



Engenharia de software e modelagem II

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

Quem sou eu?



- **Engenheiro de software Sênior** com mais de 15 anos de experiência
 - **Professor no ensino superior** desde 2014 pela:
 - Grupo Kroton (Anhanguera)
 - Fatec – Jales
 - IFSP - Votuporanga
 - **Bacharel em Sistemas de Informação** pela F.E.F
 - **Pós graduado em gestão em Banco de Dados**
 - **Mestre em Engenharia de Software** pela Universidade Federal do Pampa RS
-

E vocês quem são?



Projeto Integrador da Disciplina

Ao longo da disciplina, cada equipe deverá conceber e documentar um sistema de software fictício, aplicando os principais conceitos de Engenharia de Software II.

1. Engenharia de Requisitos

- Identificação do problema
- Stakeholders
- Levantamento e especificação de requisitos



1º
Etap
a

Projeto Integrador da Disciplina

Ao longo da disciplina, cada equipe deverá conceber e documentar um sistema de software fictício, aplicando os principais conceitos de Engenharia de Software II.

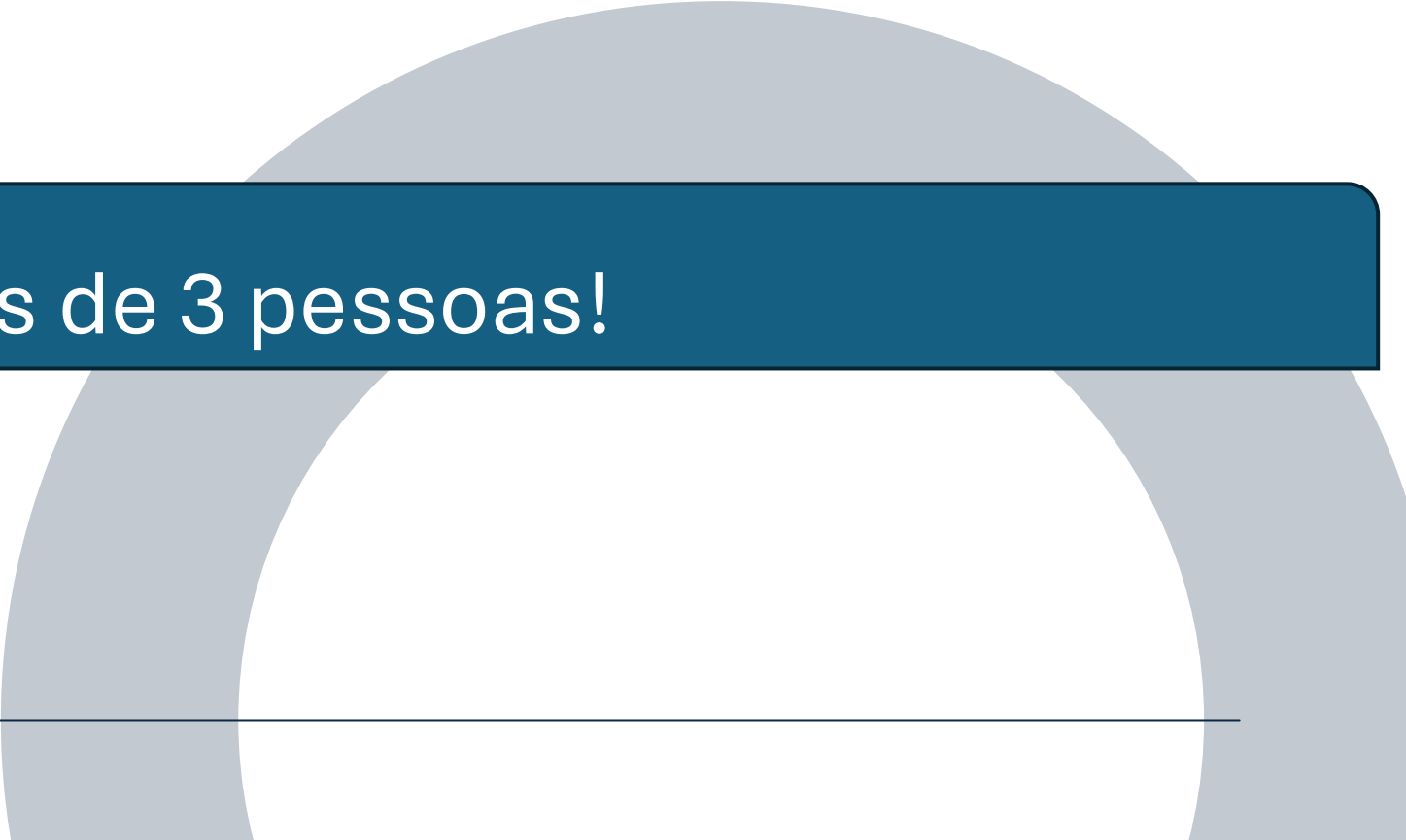
2. Arquitetura e Projeto

- Definição da arquitetura
- Padrões de projeto
- Componentização e reutilização
- Identificação de anomalias e refatoração



Fina
1

Projeto Integrador da Disciplina



Equipes de 3 pessoas!



Saúde

- Sistema de agendamento de consultas
- Gestão de clínicas
- Acompanhamento de pacientes



Educação

- Plataforma de cursos
- Sistema de gestão acadêmica
- Ambiente de avaliação e atividades



Logística

- Rastreamento de entregas
- Gestão de transportadoras
- Otimização de rotas



Comércio

- Marketplace
- Sistema de pedidos
- Gestão de estoque



Serviços Públicos

- Solicitação de serviços municipais
- Gestão de iluminação pública
- Sistema de atendimento ao cidadão



Negócios

- Gestão de projetos
- Recrutamento e seleção
- Gestão financeira

Conteúdo programático

- Fase de Projeto de software
 - Engenharia de requisitos
 - Técnicas de Elicitação e Levantamento de Requisitos
 - Modelagem de requisitos
 - Arquitetura de software
 - **Entrega Fase 1**
 - Estilos e padrões arquiteturais
-

Conteúdo programático

- Projeto de software e princípios de design
 - Padrões de projeto
 - Anomalias, code smells e refatoração
 - Reutilização com componentes e frameworks
 - **Entrega Fase 2**
 - Integração contínua e testes automatizados
 - Containers, gerência de configuração e liberação
-

Avaliação

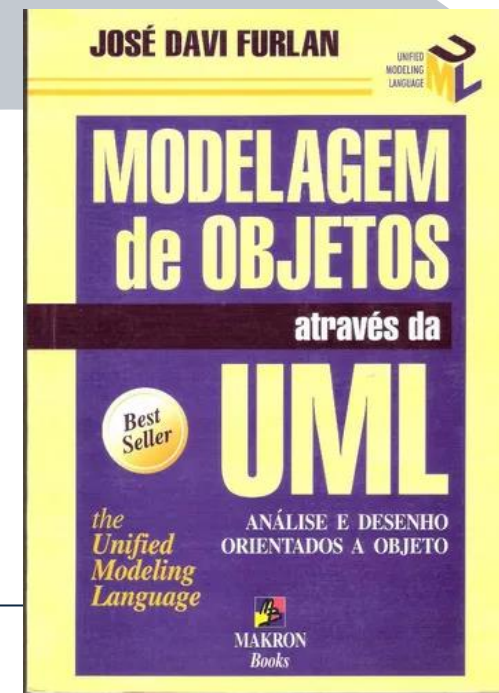
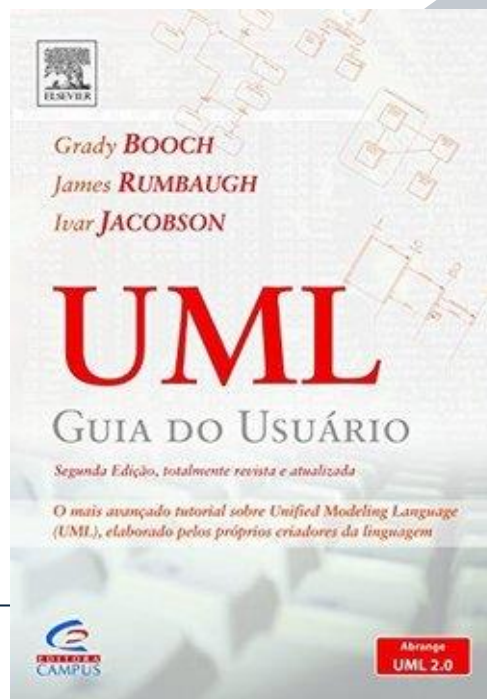
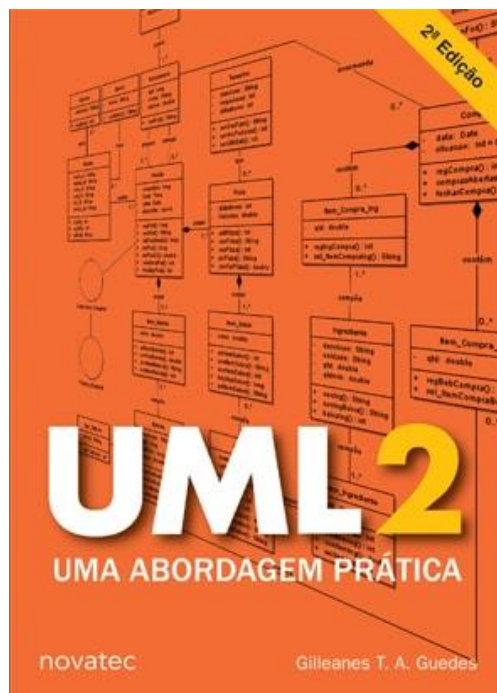
- Serão 2 avaliações (AV1 e AV2) sendo elas:
 - provas individuais com questões teóricas e práticas, constando de questões dissertativas, múltipla escolha
 - Um projeto de software (PJ) desenvolvido em grupo com papéis definidos
 - Fórmula: $[(AV1 * 0.6) + (PJ * 0.4)] + [(AV2 * 0.6) + (PJ * 0.4)] / 2$
-

Bibliografia

GUEDES, GILLEANES T. A. **UML 2 UMA ABORDAGEM PRATICA**. 2 ED. SAO PAULO: NOVATEC, 2011

BOOCH, GRADY. **UML GUIA DO USUARIO**. 1 ED. RIO DE JANEIRO: CAMPUS, 2000

FURLAN, JOSE DAVI. **MODELAGEM DE OBJETOS ATRAVES DA UML**. 1 ED. SAO PAULO: MAKRON BOOKS, 1998



Bibliografia

- BEZERRA, EDUARDO. **PRINCÍPIOS DE ANÁLISE E PROJETO DE SISTEMAS COM UML**. 4. ED. RIO DE JANEIRO: EDITORA ELSEVIER, 2007
 - MEDEIROS, ERNANI. **DESENVOLVENDO SOFTWARE COM UML 2.0**. 1. ED. SÃO PAULO: EDITORA PEARSON MAKRON BOOKS, 2004
 - O'NEIL, HENRIQUE. NUNES, MAURO. RAMOS, PEDRO. **EXERCÍCIOS DE UML**. 1.ED: EDITORA INFORMÁTICA, 2010.
 - GÓES, WILSON MORAES. **APRENDA UML POR MEIO DE ESTUDO DE CASO**. 1. ED. SÃO PAULO: EDITORA NOVATEC, 2014
 - BORATTI, ISAIAS CAMILO. **PROGRAMAÇÃO ORIENTADA A OBJETOS EM JAVA**. 1. ED. FLORIANÓPOLIS: EDITORA VISUAL BOOKS, 2007
-

Mas o que é um projeto de software?

- Software é um componente essencial para qualquer tipo de negócio
- MAS... Software não é feito em pastelaria

FALAR NÃO ADIANTA ENTÃO
VAMOS **DESENHAR:**

ISTO É UM PASTEL
QUE PODE SER FEITO EM ATÉ 3MIN.



ISTO É UM PROJETO
QUE NEM QUE VOCÊ QUEIRA PODE SER FEITO EM ATÉ 3 MIN.



Fases de um Projeto de Software

Da identificação de um problema à entrega de uma solução

- Um projeto de software passa por diferentes fases que ajudam a transformar uma necessidade em um sistema funcional, confiável e sustentável.

O ciclo do projeto



1. Identificação do Problema

Todo software começa com uma necessidade

Antes de pensar em telas, tecnologias ou código, é necessário compreender:

- Qual problema precisa ser resolvido?
 - Quem enfrenta esse problema?
 - Como o problema é resolvido atualmente?
 - Quais são as principais dificuldades?
 - Qual valor o software deverá gerar?
-

1. Identificação do Problema

Todo software começa com uma necessidade

Problema: Pequenos produtores têm dificuldade para controlar suas vendas e estoques.

Possível solução: Sistema de gestão de vendas e estoque.

Um bom software começa com um problema bem compreendido.

2. Levantamento e Engenharia de Requisitos

Descobrendo o que o sistema precisa fazer

Nesta fase, a equipe busca compreender as necessidades dos usuários e do negócio.

Principais atividades

- Identificação dos stakeholders;
- Entrevistas;
- Questionários;
- Observação;
- Workshops;
- Brainstorming;
- Prototipação;
- Análise de documentos.

3. Análise e Especificação de Requisitos

Transformando necessidades em uma especificação

Os requisitos precisam ser analisados, organizados e documentados.

Tipos de requisitos

- **Requisitos Funcionais**
 - O que o sistema deve fazer.
- **Requisitos Não Funcionais**
 - Como o sistema deve se comportar.

Exemplos

- **Funcional:** O sistema deve permitir cadastrar produtos.
- **Não funcional:** A consulta de produtos deve responder em até 2 segundos.

4. Planejamento do Projeto

Como o software será desenvolvido?

O planejamento define como a equipe organizará o trabalho.

Elementos importantes

- Escopo;
- Cronograma;
- Atividades;
- Prioridades;
- Equipe;
- Recursos;
- Riscos;
- Entregas.

Perguntas fundamentais

- **O que será desenvolvido?**
- **Quem fará?**
- **Quando será entregue?**
- **Quais riscos podem afetar o projeto**

5. Arquitetura de Software

Definindo a estrutura da solução

A arquitetura define as principais decisões estruturais do sistema.

Decisões arquiteturais

- Organização dos componentes;
- Comunicação entre módulos;
- Persistência de dados;
- Integração com sistemas externos;
- Segurança;
- Escalabilidade;
- Disponibilidade.

Exemplos de estilos arquiteturais

- Arquitetura em camadas;
- MVC;
- Hexagonal;
- Clean Architecture;
- Microservices;
- Event-Driven.

6. Projeto Detalhado

Como os componentes serão construídos?

Nesta fase, a arquitetura é detalhada em elementos que poderão ser implementados.

São definidos:

- Classes;
- Interfaces;
- Módulos;
- Componentes;
- Serviços;
- Responsabilidades;
- Dependências.

Princípios importantes

- Alta coesão;
- Baixo acoplamento;
- Encapsulamento;
- Separação de responsabilidades;
- SOLID.

Padrões de Projeto

Strategy

Factory

Observer

Adapter

Facade

7. Implementação

Transformando o projeto em código

Nesta fase, os desenvolvedores implementam a solução definida nas etapas anteriores.

Atividades

- Desenvolvimento de funcionalidades;
- Criação de componentes;
- Integração entre módulos;
- Implementação de regras de negócio;
- Acesso a dados;
- Integração com APIs.

Boas práticas

- Controle de versão;
- Revisão de código;
- Padronização;
- Testes automatizados;
- Commits frequentes.

8. Testes e Garantia da Qualidade

O software funciona como esperado?

Os testes buscam identificar problemas antes que o software chegue aos usuários.

Tipos de testes

- **Testes Unitários**
Verificam pequenas unidades de código.
- **Testes de Integração**
Verificam a comunicação entre componentes.
- **Testes End-to-End**
Validam o sistema de ponta a ponta.

9. Integração, Configuração e Entrega

Como garantir que o software possa ser entregue?

O software precisa ser integrado, empacotado e disponibilizado de forma controlada.

Práticas importantes

- Integração Contínua;
- Builds automatizados;
- Gerência de configuração;
- Controle de versões;
- Variáveis de ambiente;
- Contêineres;
- Pipelines.

Commit

Build

Testes

Análise

Empacotamento

Deploy



10. Operação, Manutenção e Evolução

O software continua depois da entrega

Após a disponibilização do sistema, novas necessidades e problemas surgem.

Atividades

- Correção de defeitos;
- Melhorias;
- Novas funcionalidades;
- Monitoramento;
- Atualizações;
- Refatoração;
- Evolução arquitetural.

Feedback

Novos
Requisitos

Desenvolvi
mento

Testes

Nova
Entrega



Resumidamente!

Faça a coisa certa (análise)

e faça certo a coisa (projeto)



Até a próxima!